

Caio Collino Troiano  
Gabriel Rosa Leite Barroso  
Lucas da Mata Guimarães  
Thiago Pereira Lins da Silva  
Vinicius Vianna Egydio de Carvalho

## Trabalho Final - Análise de Algoritmos

São Paulo  
2026

# Sumário

|     |                                  |   |
|-----|----------------------------------|---|
| 1   | ROTTING ORANGES . . . . .        | 2 |
| 1.1 | Estratégia de Solução . . . . .  | 2 |
| 2   | NEAREST EXIT FROM MAZE . . . . . | 3 |
| 2.1 | Estratégia de Solução . . . . .  | 3 |
| 3   | ICE STATUES FESTIVAL . . . . .   | 5 |
| 3.1 | Estratégia de Solução . . . . .  | 5 |
| 4   | ROUTE CHANGE . . . . .           | 7 |
| 4.1 | Estratégia de Solução . . . . .  | 7 |

# 1 Rotting Oranges

**Problema:** Rotting Oranges

**Paradigma:** Grafos (Busca em Largura / BFS Multi-source)

**Implementação:** (inserir o link pra qnd tiver o git)

## 1.1 Estratégia de Solução

O problema foi resolvido modelando a matriz  $m \times n$  como um **grafo não-direcionado**, onde:

- Cada célula da matriz nas coordenadas  $(i, j)$  funciona como um vértice;
- As conexões diretas (cima, baixo, esquerda, direita) funcionam como as arestas.

Como a podridão se espalha de forma uniforme e simultânea a partir de várias fontes, a técnica ideal é a Busca em Largura Multi-Source (BFS). O uso de uma fila é essencial nessa abordagem, pois garante o processamento nível a nível, o que é fundamental para descobrirmos o tempo mínimo de infecção considerando que já existem várias laranjas podres no início. O algoritmo funciona em três etapas:

1. **Mapeamento Inicial:** O algoritmo percorre a matriz para contar o número total de laranjas frescas e insere a posição de todas as laranjas podres iniciais na fila da BFS;
2. **Propagação por Minuto:** A fila é processada nível por nível (onde cada nível completo representa 1 minuto). Para cada laranja podre retirada da fila, o algoritmo infeta os seus vizinhos frescos válidos, mudando o estado deles para podre e colocando-os na fila para o minuto seguinte;
3. **Verificação Final:** Quando a fila fica vazia, se o contador de laranjas frescas ainda for maior que zero, significa que algumas laranjas ficaram isoladas no grafo, retornando -1. Caso contrário, retorna o total de minutos decorridos.

## 2 Nearest Exit from Maze

**Problema:** Nearest Exit from Maze

**Paradigma:** Grafos (Busca em Largura / BFS Multi-source)

**Implementação:** (inserir o link pra qnd tiver o git)

### 2.1 Estratégia de Solução

O problema foi resolvido modelando a matriz  $m \times n$  (o labirinto) como um grafo não-direcionado e não-ponderado, onde:

- Cada célula livre (caractere '.') da matriz nas coordenadas  $(i, j)$  funciona como um vértice;
- As conexões diretas (cima, baixo, esquerda, direita) entre células livres adjacentes funcionam como as arestas de peso unitário.

Como o objetivo principal é encontrar o caminho mais curto (menor número de passos) de um ponto de origem até à borda do labirinto, a técnica ideal é a Busca em Largura (BFS). O uso de uma fila (queue) é essencial nesta abordagem, pois garante uma expansão em "ondas" (nível a nível). Isto é fundamental para o problema, pois assegura que a primeira célula de saída encontrada seja, garantidamente, a mais próxima. O algoritmo funciona em três etapas principais:

1. **Mapeamento Inicial:** O algoritmo recolhe as coordenadas da entrada (entrance) e insere-as na fila da BFS com uma contagem de zero passos. Para otimizar o espaço espacial (evitando criar uma matriz extra de visitados), a própria célula de entrada é imediatamente marcada no labirinto original como visitada (mudando o caractere para '+'). Note-se que a célula de entrada, mesmo que esteja na borda, não conta como saída válida;
2. **Busca e Expansão (Passo a Passo):** A fila é processada vértice a vértice. Para cada célula retirada, o algoritmo calcula as posições dos seus quatro vizinhos. Se um vizinho for válido (estiver dentro dos limites e for um caminho livre '.'), o algoritmo marca-o como visitado ('+'). Imediatamente a seguir, verifica se essa célula vizinha se encontra numa das bordas da matriz (linha ou coluna igual a 0, ou igual ao limite máximo). Se for uma borda, a saída mais próxima foi encontrada e o algoritmo retorna de imediato o número de passos atuais mais um. Caso contrário, coloca o vizinho na fila para a expansão do próximo nível;
3. **Verificação Final:** Quando a fila de processamento fica completamente vazia, significa que todos os caminhos possíveis foram explorados. Se a execução chegar a este ponto

sem ter retornado nenhum valor na etapa anterior, conclui-se que não existe nenhuma rota possível para uma saída, retornando assim  $-1$ .

## 3 Ice Statues Festival

**Problema:** Ice Statues Festival

**Paradigma:** Programação Dinâmica (Bottom-Up / Tabulação)

**Implementação:** (inserir o link pra qnd tiver o git)

### 3.1 Estratégia de Solução

O problema foi resolvido modelando a busca pela quantidade mínima de blocos de gelo para atingir o comprimento desejado  $M$  como uma tabela de estados de otimização (idêntico ao problema clássico Coin Change), onde:

- Cada índice  $i$  do vetor (tabela) funciona como um subproblema, representando um comprimento específico de bloco que se deseja formar;
- O valor armazenado em cada índice  $i$  representa a quantidade mínima de blocos de gelo necessários para formar aquele comprimento;
- Como o problema exige encontrar uma resposta ótima (menor número de blocos) e as soluções para valores maiores dependem diretamente das soluções para valores menores (caracterizando **subestrutura ótima** e **sobreposição de subproblemas**), a técnica ideal é a Programação Dinâmica;
- O uso de uma tabela (vetor) construída de baixo para cima (bottom-up) é essencial nessa abordagem, pois armazena os resultados parciais e evita o recálculo redundante, garantindo eficiência mesmo para valores de  $M$  de até 1.000.000.

O algoritmo funciona em **três etapas**:

- **Inicialização da Tabela:** O algoritmo cria um vetor  $dp$  de tamanho  $M + 1$ . O caso base é definido como  $dp[0] = 0$ , estabelecendo que são necessários zero blocos para obter um comprimento de tamanho zero. Todos os outros espaços são inicializados com um valor muito grande (infinito simulado);
- **Construção Bottom-Up:** Através de laços aninhados, o algoritmo percorre a tabela de 1 até  $M$ . Para cada posição  $i$ , ele avalia todos os blocos disponíveis cujos comprimentos sejam menores ou iguais a  $i$ . Utilizando a equação de recorrência  $dp[i] = \min(dp[i], dp[i - \text{bloco}] + 1)$ , o algoritmo garante que o valor salvo seja sempre a melhor escolha;

- **Verificação Final:** Após preencher toda a tabela, a resposta ótima para o caso de teste estará no último índice ( $dp[M]$ ), bastando imprimi-la. Como o bloco de tamanho 1 sempre está presente, há a garantia de que uma solução viável sempre existirá.

## 4 Route Change

**Problema:** Route Change

**Paradigma:** Grafos (Algoritmo de Dijkstra para Caminho Mínimo com Restrições)

**Implementação:** (inserir o link pra qnd tiver o git)

### 4.1 Estratégia de Solução

O problema foi resolvido modelando o sistema de estradas do país como um **grafo valorado** e **não-direcionado**, onde:

- Cada cidade representa um vértice (identificado de 0 a  $N - 1$ );
- Cada estrada representa uma aresta com peso equivalente ao custo do pedágio;
- Como o objetivo é encontrar o custo mínimo para atingir a cidade destino ( $C - 1$ ) a partir de uma cidade de quebra ( $K$ ), considerando regras específicas para a rota de serviço fixa  $(0, 1, \dots, C - 1)$ , a técnica ideal é o **Algoritmo de Dijkstra**.

A restrição fundamental do problema diz que, sempre que o veículo interceptar qualquer cidade que pertença à rota de serviço, ele **obrigatoriamente deve seguir a rota sequencial de forma linear** até o destino final, não podendo mais sair dela. O algoritmo funciona em **três etapas**:

1. **Modelagem e Adaptação do Grafo:** O grafo é montado por meio de uma matriz de adjacência (ou lista de adjacência). Para simplificar a restrição da rota de serviço, as arestas que partem de qualquer cidade  $u$  pertencente à rota ( $0 \leq u < C$ ) são modificadas: ela só possuirá uma única saída válida direcionada para a cidade seguinte da rota ( $u + 1$ ), com o peso correspondente ao pedágio dessa rota linear. Qualquer outra estrada saindo de  $u$  é ignorada, pois o veículo não pode desviar se já estiver na rota;
2. **Processamento do Caminho Mínimo (Dijkstra):** Executa-se o algoritmo de Dijkstra partindo do vértice inicial  $K$  (a cidade onde o veículo foi consertado). Utiliza-se um vetor de distâncias `dist` inicializado com infinito e uma fila de prioridades (ou busca linear do menor vértice não visitado devido aos limites moderados de  $N \leq 250$ ) para computar o menor custo cumulativo acumulado até cada vértice do grafo;
3. **Verificação Final:** Quando o algoritmo termina de processar os caminhos, o valor mínimo do custo acumulado para alcançar de forma válida o destino estará contido na posição correspondente à última cidade da rota de serviço (`dist[C - 1]`), sendo impresso como resultado para cada caso de teste.